

Chanwu (Tyler) Oh

Cloud backend developer who brings AI into services through harness engineering. Bilingual in English and Korean, with a strong interest in cross-cultural technical communication.

PROJECTS

Job Scraper

Job scraper, AI fit scoring

BE: Go | FE: JS, CSS, HTML

May 2026 - Present

Goal: Collect junior developer **job posts in one place and calculate user-specific fit scores with AI inference**

Problem: In the current local scraping model, each user repeats the same requests to the same job portals

Analysis: Fit scoring is better handled locally, while post collection and normalization do not need to repeat per user

Solution: Split the system so **scraping and normalization run on AWS, while user-specific fit scoring stays local**

Forecast: Job portal request volume drops from $O(\text{users} \times \text{posts}) \rightarrow O(\text{posts})$

Theory: Keeping user-specific scoring local and moving shared scraping work to the cloud reduces repeated requests and infrastructure cost.

Cha Physical Therapy

Web app for an NYC physical therapy clinic

Deploy: Docker, AWS, Terraform, Cloudflare | BE: Go | FE: JS, CSS, HTML

Aug 2025 - Apr 2026

Squarespace → AWS Migration

Goal: Improve the UI and implement **Stripe payments and lower latency**

Problem: Squarespace SaaS limited UI customization, and custom CSS/JS code injections caused slow page response times

Analysis: A working demo with improved UI and speed would persuade the marketing team better than explanation alone

Solution: Built a GitHub Pages demo to **make the case for moving to AWS**, then rebuilt the app with a Go backend and Docker deployment

Lesson: For migrations and new features, a **working prototype** is often the most persuasive argument

Crash Recovery and Service Hardening (details: postmortem)

Problem: In April 2026, the service failed after exceeding the I/O throughput limit of an AWS EBS gp3 volume

Analysis: Accumulated data, inefficient queries, and missing timeouts caused disk I/O to spike

Solution: Applied **query rewrites, indexes, timeouts, data cleanup jobs, and CloudWatch monitoring**

Lesson: Database optimization and monitoring are core to reliability, not just performance

- Goal:** Build a French architect portfolio that works consistently across **desktop, tablet, and mobile**
- Problem:** Large image files slowed loading, and the layout needed to hold across screen sizes
- Analysis:** Because the client did not want domain costs, a GitHub Pages static site was the best fit
- Solution:** Built a **responsive layout with CSS breakpoints and JavaScript**, and converted images to WebP to reduce size
- Lesson:** Hand-building responsive UI strengthened my fundamentals and showed me the value of declarative frameworks like React

SKILLS

- Languages: **Go** **Python** **JS**
- Database: **MySQL** **MongoDB**
- Deploy: **AWS** **Docker** **Terraform** **Cloudflare**
- Tools: **Git**

OPEN SOURCE

- freeCodeCamp** *JS Curriculum*
- Corrected inaccurate content in the WeakSet lesson

EDUCATION

- Hanyang University** *Seoul, South Korea*
- International Studies, B.A.** *2017-2025*
- GPA: 3.91 / 4.5 (Graduated with honors)

CERTIFICATION

- TOEFL** *119 / 120*
- C2 - Native

MILITARY SERVICE

- Second Operational Command** *Daegu, South Korea*
- English-Korean Interpreter** *2020-2022*